CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 11

Student Management in the Admin Console – CRUD, Search, Filters, Soft Delete and CSV Export

PHP 8 · MySQLi prepared statements · MariaDB (studenthub_db) · Bootstrap 5 Admin Console

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 11 report (`Practical-11/PRAC_11_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report describes a standalone application in `Practical-11/` with its own database `studenthub_prac11` and no login. This corrected version describes the same requirements as they are now implemented in the Student Records tab of the Admin Console of the integrated portal (database `studenthub_db`, one login), including the later fix that a deactivated student can no longer sign in. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To implement complete student management with PHP, MySQL and MySQLi that performs all record operations safely and keeps every record, even after a student is removed from the active list – as part of the one integrated StudentHub portal, working on the same student records that self-registration, the profile pages and the event registrations use.

- Display the student records of the portal database in a paginated table (10 per page) with a clear Active / Inactive status and whether the record is linked to a login account.
- Add and update students with server-side validation, a unique enrollment number and e-mail address, and clear messages.
- Search by name, e-mail and enrollment number, and filter by course, semester and status – all combinable.
- Soft delete a student only after an explicit confirmation: the status changes from Active to Inactive and the row stays in MySQL together with its account link and event registrations; a Reactivate button restores it.
- Export the filtered records to a CSV file that opens correctly in Microsoft Excel and is protected against formula injection.
- Use MySQLi prepared statements for every query with values, a CSRF token on every POST, output escaping, administrator-only access and an audit-log entry for every change.
- Integration requirement: a student account whose linked record is deactivated can no longer sign in, and an open session of that student ends at its next request.

2. Problem Definition

An institute's student list changes constantly: students join, change course or semester, and leave. Staff must be able to find a student quickly and export the list for spreadsheet work. At the same time a record must never be lost by a wrong click, so removing a student must be reversible – a soft delete that only marks the student inactive. Every input comes from the browser and is therefore untrusted: it has to be validated on the server, sent to MySQL only through prepared statements, and escaped whenever it is shown again.

Inside the integrated portal there is an additional requirement. The table `studenthub_db.students` already holds the records created by self-registration (Practical 9, linked to the login account through `students.user_id`) and is referenced by `registrations` (Practical 8 foundation) with `ON DELETE CASCADE`. A separate Practical 11 database would create a second, diverging student list, and a hard delete would silently remove a student's event-registration history. Student management therefore had to be built on the existing table: enrollment numbers keep the existing `YYPPnnn` scheme (e.g. `25CS011`), the existing status values `Active` / `Inactive` are reused, and no student row is ever deleted by the portal.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Admin Console page, validation, sessions, CSV generation
MySQLi (exception mode)	Every query with values as a prepared statement (`prepare` / `bind_param` / `execute`)
MariaDB 10.4.32 (XAMPP)	Shared portal database `studenthub_db`; migration 002 (unique enrollment numbers)
Bootstrap 5.3.2 + Bootstrap Icons	Layout of the Admin Console (loaded from the jsDelivr CDN)
HTML5 forms (`novalidate`)	Form fields and messages; the server alone decides whether input is valid
Python 3	`tests/test_students.py` (HTTP tests) and `database/tests/test_002_unique_enrollment_no.py` on disposable databases
Microsoft Edge (headless) + Playwright	`tests/browser_students.py` and `tests/browser_inactive_login.py` – browser checks and the screenshots
Git / GitHub	Commits `0ac2f5d` (integration) and `6420efc` (sign-in fix), pushed to the project repository

4. Place in the Integrated Portal

Practical 11 is no longer a separate application. It is the **Student Records (CRUD)** tab of the Admin Console (`studenthub/admin_dashboard.php?tab=students`), which an administrator reaches after signing in at the portal's single login page. The logic lives in one include file; the console page only calls it.

File / route	Role in Practical 11
`admin_dashboard.php?tab=students`	List with search, filters and pagination; `view=add`, `view=edit&id=N` and `view=deactivate&id=N` only *show* the forms (GET never changes data)
`admin_dashboard.php` (POST)	`action=add_student`, `update_student`, `deactivate_student`, `reactivate_student` – CSRF token and permission `students.manage` required
`admin_dashboard.php?tab=students&export=csv`	CSV download of the records matching the current search and filters (read-only)
`includes/student_management.php`	Filters, list, validation, duplicate checks, save, soft delete / reactivation, CSV export
`includes/registration.php`	Course list `REGISTRATION_COURSES` and `next_enrollment_number()`, shared with self-registration
`includes/auth.php`, `login.php`	Login, session, `require_permission()`, CSRF; `account_is_deactivated()` / `account_block_reason()`
`includes/event_registration.php`, `directory.php`	Inactive students cannot register for events and are not listed in the student directory
`audit_logger.php`	`log_audit()`: CREATE_STUDENT, UPDATE_STUDENT, DEACTIVATE_STUDENT, REACTIVATE_STUDENT, EXPORT_CSV
`database/migrations/002_unique_enrollment_no.sql`	UNIQUE index on `students.enrollment_no` (with `.down.sql` rollback)

Shared services. The tab uses the portal's one authentication system (session cookie `STUDENTHUBSESSID`, HttpOnly, SameSite=Strict), the one CSRF token of the session, the common HTTP 403 page and the common audit log (shown in the console's *Security Audit Trail* tab). The permissions are listed explicitly in `includes/auth.php`: `admin.console` and `students.manage` belong to the role `admin` only, so students and editors receive HTTP 403 for the tab, the forms, the export and every POST action, and each refusal is logged as ACCESS_DENIED.

5. Database Design and Relationships

Practical 11 uses the portal table `students` (InnoDB, `utf8mb4_unicode_ci`). Its structure, as created by `database/studenthub_demo.sql`, is:

Column	Type	Notes
id	INT(11) AUTO_INCREMENT	Primary key (record ID)
user_id	INT(11) NULL	Link to the login account; UNIQUE `uq_students_user_id`, FK `fk_students_user` → `users.id` ON DELETE SET NULL
enrollment_no	VARCHAR(20) NULL	UNIQUE `uq_students_enrollment_no` (migration 002); NULL allowed for legacy rows
name	VARCHAR(100) NOT NULL	Full name
email	VARCHAR(100) NOT NULL	UNIQUE key `email`
course	VARCHAR(50) NOT NULL	Course name; existing names are kept
gender	VARCHAR(10) NOT NULL	Male / Female / Other for new records
semester	INT(11) DEFAULT 3	CHECK `chk_students_semester`: NULL or 1–8
status	VARCHAR(20) DEFAULT 'Active'	Active / Inactive – the soft-delete flag
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP

5.1 Relationships

Relationship	Rule	Effect for student management
users 1 : 0..1 students	`students.user_id` UNIQUE, ON DELETE SET NULL	A record can be linked to one login account; the Admin Console never changes the link
students 1 : n registrations	`fk_registrations_student` ON DELETE CASCADE	Deleting a student would erase registrations – the portal therefore never deletes students (soft delete)
events 1 : n registrations	`fk_registrations_event` ON DELETE CASCADE; UNIQUE (student_id, event_id)	A student registers at most once per event
users 1 : n audit_logs	`audit_logs.user_id` ON DELETE SET NULL	Every change made in the tab is logged with the administrator's account

5.2 Migration 002 – unique enrollment numbers

Because the console now adds and edits records, two records must never share an enrollment number. The application checks it, and migration 002 makes the database enforce it as well (also for simultaneous requests). The script first checks its preconditions and changes nothing if one fails: migration 001 is recorded, the column exists and no number is used twice (the column collation is case-insensitive, so `25cs011` and `25CS011` count as the same number). It then adds the index, verifies it and records version `002` in `schema_migrations`. It is idempotent; `002_unique_enrollment_no.down.sql` drops only the index and the `002` row. According to `database/README.md` it was applied to the live database on 31 Oct 2026 after a verified backup (9 records, no duplicates).

database/migrations/002_unique_enrollment_no.sql (lines 40–51) – duplicate precondition and the UNIQUE index

```
40      -- the column collation is case-insensitive, so "25cs001" and "25CS001" count as the same number
41      IF EXISTS (SELECT 1 FROM students WHERE enrollment_no IS NOT NULL GROUP BY enrollment_no HAVING
    COUNT(*) > 1) THEN
42          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 002: some enrollment numbers are used by
    more than one student. Correct them first (see database/README.md).';
43      END IF;
44  END //
45
46  DELIMITER ;
47
48  -- ------------------------------------------------------------------------------------------------------
    -----------------
49  -- 1. The unique index
50  -- ------------------------------------------------------------------------------------------------------
    -----------------
51  ALTER TABLE students ADD UNIQUE KEY IF NOT EXISTS uq_students_enrollment_no (enrollment_no);
```

Soft delete in the schema. No column was added for the soft delete. Deactivating a student sets the existing `status` column to `Inactive`; the row, its `user_id` link and its registrations stay. The time of the deactivation is recorded in the audit log (DEACTIVATE_STUDENT), not in a column of `students`.

6. Implementation

6.1 Search, filters and pagination

`student_filters()` reads the query string and accepts a course, semester or status only if it is one of the known values (unknown values are ignored, never used in SQL). `student_filter_sql()` builds one WHERE clause with `?` parameter markers that is shared by the table, the page count and the CSV export. The search text is matched against name, e-mail and enrollment number; `%` and _ typed by the user are escaped with `addcslashes()` so they are searched literally. The list is ordered newest first and shows 10 records per page; page links keep the filters.

studenthub/includes/student_management.php (lines 72–98) – one parameterised WHERE clause for list, count and export

```
72  function student_filter_sql(array $filters): array
73  {
74      $where = [];
75      $types = '';
76      $params = [];
77      if ($filters['q'] !== '') {
78          $like = '%' . addcslashes($filters['q'], '\\%_') . '%';
79          $where[] = '(name LIKE ? OR email LIKE ? OR enrollment_no LIKE ?)';
80          $types .= 'sss';
81          array_push($params, $like, $like, $like);
82      }
83      if ($filters['course'] !== '') {
84          $where[] = 'course = ?';
85          $types .= 's';
86          $params[] = $filters['course'];
87      }
88      if ($filters['semester'] !== '') {
89          $where[] = 'semester = ?';
90          $types .= 'i';
91          $params[] = (int) $filters['semester'];
92      }
93      if ($filters['status'] === 'Inactive') {
94          $where[] = "TRIM(status) = 'Inactive'";
95      } elseif ($filters['status'] === 'Active') {
96          $where[] = "COALESCE(TRIM(status), '') <> 'Inactive'";
97      }
98      return [$where ? ' WHERE ' . implode(' AND ', $where) : '', $types, $params];
```

6.2 Add and edit with server-side validation

`validate_student_record()` checks every field: the enrollment number must match `^[0-9]{2}[A-Z]{2}[0-9]{3}$` (stored in upper case), the name 2–100 letters (spaces, apostrophes, dots and hyphens allowed, spaces collapsed), a valid e-mail of at most 100 characters (stored in lower case), a course from the list, a semester 1–8 and the gender Male, Female or Other; a field sent as an array is refused. When a record is edited, an unchanged legacy value (an old number format, course or gender, or no semester) is kept as it is. `check_student_record()` then adds the duplicate checks, so a duplicate number is reported together with all other errors in one answer (HTTP 422, the typed values shown again HTML-escaped).

studenthub/includes/student_management.php (lines 254–283) – validation plus duplicate checks (all problems at once)

```
254  function check_student_record(mysqli $db, array $input, array $courses, ?array $existing, int $id):
     array
255  {
256      [$clean, $errors] = validate_student_record($input, $courses, $existing);
257      $conflicts = student_conflicts($db, isset($errors['enrollment_no']) ? null :
     $clean['enrollment_no'],
258                                     isset($errors['email']) ? null : $clean['email'], $id);
259      return [$clean, $errors + $conflicts];
260  }
261
262  // Other records that already use this enrollment number or e-mail (any status – inactive records
     keep theirs)
263  function student_conflicts(mysqli $db, ?string $enrollment, ?string $email, int $except_id): array
264  {
265      $errors = [];
266      if ($enrollment !== null) {
267          $stmt = $db->prepare('SELECT 1 FROM students WHERE enrollment_no = ? AND id <> ? LIMIT 1');
268          $stmt->bind_param('si', $enrollment, $except_id);
269          $stmt->execute();
270          if ($stmt->get_result()->num_rows > 0) {
271              $errors['enrollment_no'] = 'This enrollment number is already used by another student.';
272          }
273      }
274      if ($email !== null) {
275          $stmt = $db->prepare('SELECT 1 FROM students WHERE LOWER(TRIM(email)) = ? AND id <> ? LIMIT
     1');
276          $lower = strtolower(trim($email));
277          $stmt->bind_param('si', $lower, $except_id);
278          $stmt->execute();
279          if ($stmt->get_result()->num_rows > 0) {
280              $errors['email'] = 'Another student record already uses this email address.';
281          }
282      }
283      return $errors;
```

`save_student_record()` writes the record in a transaction. If the enrollment number was left empty on the add form, the next free number of the existing scheme is assigned: two digits of the admission year (derived from the semester and the academic year) plus the course code from `REGISTRATION_COURSES` (IT, CS, DS, WD) and a three-digit counter, computed by `next_enrollment_number()`, which locks the matching rows with `SELECT … FOR UPDATE` until the commit. The duplicate checks run again inside the transaction, and a duplicate saved by someone else at the same moment is caught by the UNIQUE keys (MySQL error 1062) and turned into a field message. Only the form fields are written: `user_id`, `status`, `created_at` and the registrations are never touched, and no login account is created.

studenthub/includes/student_management.php (lines 288–321) – add / update in one transaction

```
288  function save_student_record(mysqli $db, array $clean, ?int $id = null): array
289  {
290      $db->begin_transaction();
291      try {
292          $enrollment = $clean['enrollment_no'];
293          if ($id === null && $enrollment === null) {
294              $prefix = sprintf('%02d', student_admission_year($clean['semester']) % 100) .
     REGISTRATION_COURSES[$clean['course']];
295              $enrollment = next_enrollment_number($db, $prefix);              // locks the prefix
     range until commit
296          }
```

```
297         $errors = student_conflicts($db, $enrollment, $clean['email'], $id ?? 0);
298         if ($errors) {
299             $db->rollback();
300             return ['errors' => $errors];
301         }
302         if ($id === null) {
303             $stmt = $db->prepare("INSERT INTO students (enrollment_no, name, email, course, semester,
    gender, status) VALUES (?, ?, ?, ?, ?, ?, 'Active')");
304             $stmt->bind_param('ssssis', $enrollment, $clean['name'], $clean['email'],
    $clean['course'], $clean['semester'], $clean['gender']);
305             $stmt->execute();
306             $id = (int) $stmt->insert_id;
307         } else {
308             $stmt = $db->prepare('UPDATE students SET enrollment_no = ?, name = ?, email = ?, course
    = ?, semester = ?, gender = ? WHERE id = ?');
309             $stmt->bind_param('ssssisi', $enrollment, $clean['name'], $clean['email'],
    $clean['course'], $clean['semester'], $clean['gender'], $id);
310             $stmt->execute();
311         }
312         $db->commit();
313         return ['id' => $id, 'enrollment_no' => $enrollment];
314     } catch (mysqli_sql_exception $e) {
315         $db->rollback();
316         if ($e->getCode() === 1062) {                       // the same number or e-mail was saved by
    someone else meanwhile
317             return ['errors' => str_contains($e->getMessage(), 'uq_students_enrollment_no')
318                 ? ['enrollment_no' => 'This enrollment number is already used by another student.']
319                 : ['email' => 'Another student record already uses this email address.']];
320         }
321         throw $e;
```

6.3 Soft delete and reactivation

The Deactivate button only opens `admin_dashboard.php?tab=students&view=deactivate&id=N`, a confirmation panel that shows the record, the number of its event registrations and the consequences. Only the button *Yes, deactivate student* sends a POST with the CSRF token. `set_student_status()` changes nothing but the status; the condition in the WHERE clause makes a second attempt harmless ("already inactive" / "already active").

studenthub/includes/student_management.php (lines 327–343) – status-only soft delete and the CSV cell guard

```
327 function set_student_status(mysqli $db, int $id, bool $active): bool
328 {
329     $stmt = $db->prepare($active
330         ? "UPDATE students SET status = 'Active' WHERE id = ? AND TRIM(status) = 'Inactive'"
331         : "UPDATE students SET status = 'Inactive' WHERE id = ? AND COALESCE(TRIM(status), '') <>
    'Inactive'");
332     $stmt->bind_param('i', $id);
333     $stmt->execute();
334     return $stmt->affected_rows === 1;
335 }
336
337 // ------------------------------------------------------------------ CSV export
338 // A cell starting with = + - @, tab or carriage return gets a leading apostrophe, so a spreadsheet
    shows it as text
339 // instead of running it as a formula.
340 function csv_safe_cell($value): string
341 {
342     $text = (string) $value;
343     return $text !== '' && strpbrk($text[0], "=+-@\t\r") !== false ? "'" . $text : $text;
```

The POST handler of the console checks the CSRF token first, then the permission, loads the record and calls the functions above; every successful change is written to the audit log and answered with a redirect (Post/Redirect/Get) and a one-time success message.

studenthub/admin_dashboard.php (lines 68–106) – POST handler of the Student Records tab

```
68 if ($_SERVER["REQUEST_METHOD"] === "POST") {
69     require_valid_csrf();
70     $action = $_POST["action"] ?? "";
71
```

```
 72      // Student records (Practical 11): add, edit, soft delete and reactivation – admins only
    (students.manage)
 73      if (in_array($action, ["add_student", "update_student", "deactivate_student",
    "reactivate_student"], true)) {
 74          require_permission("students.manage");
 75          $current_tab = "students";
 76          $target_id = student_id_from($_POST["id"] ?? "");
 77          $record    = $target_id > 0 ? find_student_record($conn, $target_id) : null;
 78          if ($action !== "add_student" && $record === null) {
 79              flash("error", "That student record was not found.");
    redirect("admin_dashboard.php?tab=students");
 80          }
 81          if ($action === "add_student" || $action === "update_student") {
 82              $existing = $action === "update_student" ? $record : null;
 83              [$clean, $errors] = check_student_record($conn, $_POST, $student_courses, $existing,
    $existing ? $target_id : 0);
 84              $result = $errors ? ["errors" => $errors] : save_student_record($conn, $clean, $existing
    ? $target_id : null);
 85              if (isset($result["errors"])) {
 86                  http_response_code(422);
 87                  $student_form = ["mode" => $existing ? "edit" : "add", "id" => $target_id, "record"
    => $existing, "errors" => $result["errors"],
 88                                   "values" => array_map(fn ($field) => is_string($_POST[$field] ??
    null) ? $_POST[$field] : "", array_combine(STUDENT_FIELDS, STUDENT_FIELDS))];
 89              } else {
 90                  $number = $result["enrollment_no"] ?? "no enrollment number";
 91                  log_audit($conn, $current_user["id"], $current_user["username"], $existing ?
    "UPDATE_STUDENT" : "CREATE_STUDENT",
 92                      ($existing ? "Updated" : "Added") . " student record ID {$result["id"]}
    ($number)", "SUCCESS");
 93                  flash("success", $existing ? "Student “{$clean["name"]}” ($number) was updated."
 94                                             : "Student “{$clean["name"]}” was added with enrollment
    number $number.");
 95                  redirect("admin_dashboard.php?tab=students");
 96              }
 97          } else {
 98              $activate = $action === "reactivate_student";
 99              $label    = "“{$record["name"]}” (" . ($record["enrollment_no"] ?: "no enrollment
    number") . ")";
100              if (set_student_status($conn, $target_id, $activate)) {
101                  log_audit($conn, $current_user["id"], $current_user["username"], $activate ?
    "REACTIVATE_STUDENT" : "DEACTIVATE_STUDENT",
102                      ($activate ? "Reactivated" : "Deactivated (soft delete)") . " student record ID
    $target_id (" . ($record["enrollment_no"] ?: "no enrollment number") . ")", "SUCCESS");
103                  flash("success", $activate ? "Student $label was reactivated."
104                                             : "Student $label was deactivated. The record, its account
    link and its event registrations are kept; a linked student account can no longer sign in.");
105              } else {
106                  flash("error", "Student $label is already " . ($activate ? "active." : "inactive."));
```

6.4 CSV export

The *Export CSV* button carries the current search and filters (it reads *Export CSV (filtered)* when a filter is active). The export calls `student_list()` without a limit, so every matching record of every page is written, inactive ones included. The file `studenthub_students_<date>_<time>.csv` is UTF-8 with a byte-order mark, comma separated, with CRLF line endings, so Excel opens it directly. It contains only record fields – the account column says only *Linked* or *None*; no password hash, role, token or session data is exported. Every cell passes `csv_safe_cell()` (shown in 6.3): a value starting with =, +, -, @, tab or carriage return gets a leading apostrophe so that a spreadsheet shows it as text instead of running it as a formula. Each export is audit-logged (EXPORT_CSV with the number of records).

studenthub/includes/student_management.php (lines 348–362) – CSV download

```
348  function send_students_csv(array $students): void
349  {
350      header('Content-Type: text/csv; charset=utf-8');
351      header('Content-Disposition: attachment; filename="studenthub_students_' . date('Ymd_His') .
     '.csv"');
352      $out = fopen('php://output', 'w');
353      fwrite($out, "\xEF\xBB\xBF");
354      fputcsv($out, ['Record ID', 'Enrollment No', 'Name', 'Email', 'Course', 'Semester', 'Status',
```

```
    'Login Account', 'Created At'],
355             ',', '"', '', "\r\n");
356     foreach ($students as $s) {
357         fputcsv($out, array_map('csv_safe_cell', [
358             $s['id'], $s['enrollment_no'] ?? '', $s['name'], $s['email'], $s['course'],
    $s['semester'] ?? '',
359             student_is_active($s) ? 'Active' : 'Inactive', $s['user_id'] !== null ? 'Linked' :
    'None', $s['created_at'] ?? '',
360         ]), ',', '"', '', "\r\n");
361     }
362     fclose($out);
```

6.5 Deactivated students cannot sign in (fix, commit 6420efc)

After the first integration step a deactivated student could still sign in (listed as a limitation in TEST_REPORT §7). The fix in `includes/auth.php` and `login.php` treats a `student` account whose linked record is `Inactive` as blocked. `account_block_reason()` also covers accounts disabled by an administrator (`users.status`, added later with Practical 14). The account and its record are re-read on every request, so an open session ends at the next request: the session data is deleted on the server, the visitor gets a new empty session ID and SESSION_REVOKED is logged. Admin and editor accounts, even if linked to an inactive record, and student accounts without a linked record are not affected.

studenthub/includes/auth.php (lines 185–224) – block rule and per-request re-check

```
185 // A student account whose linked student record (students.user_id) was deactivated in the Admin
    Console (Practical 11
186 // soft delete) may neither sign in nor keep a session. Admin and editor accounts and student
    accounts without a linked
187 // record are not affected. "Inactive" is matched like everywhere else in the portal (trimmed, any
    letter case).
188 function account_is_deactivated(string $role, ?string $student_status): bool
189 {
190     return $role === 'student' && $student_status !== null && strcasecmp(trim($student_status),
    'Inactive') === 0;
191 }
192
193 // Why an account may not sign in or keep a session (null = allowed): an account disabled by an
    administrator (users.status,
194 // Practical 14, any role) or a student account whose linked student record is inactive (Practical
    11)
195 function account_block_reason(array $row): ?string
196 {
197     if (($row['account_status'] ?? 'active') === 'disabled') {
198         return 'the account is disabled';
199     }
200     return account_is_deactivated((string) $row['role'], $row['student_status'] ?? null) ? 'the
    linked student record is inactive' : null;
201 }
202
203 // Re-reads the account on every request: a deleted account loses access at once, a role change
    applies immediately and a
204 // disabled account or deactivated student is signed out. Returns [user or null, reason
    ('account_unavailable' or
205 // 'access_disabled'), detail for the audit log].
206 function refresh_user_from_database(array $user): array
207 {
208     global $conn;
209     $stmt = $conn->prepare('SELECT u.id, u.username, u.email, u.role, u.status AS account_status,
    s.status AS student_status
210                             FROM users u LEFT JOIN students s ON s.user_id = u.id WHERE u.id = ?');
211     $stmt->bind_param('i', $user['id']);
212     $stmt->execute();
213     $row = $stmt->get_result()->fetch_assoc();
214     $stmt->close();
215     if (!$row || !in_array($row['role'], STUDENTHUB_ROLES, true)) {
216         return [null, 'account_unavailable', ''];
217     }
218     if (($block = account_block_reason($row)) !== null) {
219         return [null, 'access_disabled', $block];
220     }
221     $_SESSION['username'] = $row['username'];
222     $_SESSION['email']    = $row['email'];
```

```
223      $_SESSION['role']     = $row['role'];
224      return [['id' => (int) $row['id'], 'username' => $row['username'], 'email' => $row['email'],
     'role' => $row['role']], '', ''];
```

At sign-in the block is checked only **after** `password_verify()` succeeded. The answer is HTTP 403 with one generic message (*"Your access to StudentHub is currently disabled. Please contact the administrator."*), no session is started and LOGIN_DENIED is logged; a wrong password still gets the normal *"Invalid username/email or password."* (HTTP 401), so the status of a record is never revealed to someone who does not know the password.

studenthub/login.php (lines 38–69) – sign-in refused after password verification

```
38          // An unknown account is checked against a dummy hash, so both failures take about the same
    time
39          $hash = $account["password"] ??
    '$2y$10$cL5pLAsT9eF311SYAHI/9OCtAW62X11zkDZ3VA2wDn4DbvPI3.YlC';
40          $verified = password_verify($password, $hash);
41          // Correct password, but the account is disabled or its linked student record is deactivated:
    no session is started
42          // (generic message; the status is only revealed to someone who knows the password)
43          $block  = $account !== null && $verified ? account_block_reason($account) : null;
44          $denied = $block !== null;
45          if ($denied) {
46              log_audit($conn, (int) $account["id"], $account["username"], "LOGIN_DENIED", "Sign-in
    refused: $block.", "SECURITY_BLOCK");
47          } elseif ($account !== null && $verified && in_array($account["role"], STUDENTHUB_ROLES,
    true)) {
48              login_user($account);
49              $up = $conn->prepare("UPDATE users SET last_login = NOW() WHERE id = ?");
50              $up->bind_param("i", $account["id"]);
51              $up->execute();
52              $up->close();
53              log_audit($conn, (int) $account["id"], $account["username"], "USER_LOGIN", "User logged
    in with role=" . $account["role"], "SUCCESS");
54              if ($account["role"] === "admin") {
55                  flash("success", "Welcome, " . $account["username"] . ". You are signed in as
    administrator.");
56                  redirect("admin_dashboard.php");
57              }
58              flash("success", "Welcome back, " . $account["username"] . ".");
59              redirect("dashboard.php");
60          }
61          if ($denied) {
62              $error = LOGIN_STATUS["access_disabled"][1];
63          } else {
64              $error = "Invalid username/email or password.";
65              log_audit($conn, null, mb_substr($entered, 0, 100), "LOGIN_FAILED",
66                  $account === null ? "Sign-in attempt for an unknown account." : "Wrong password.",
    "FAILED");
67          }
68      }
69      http_response_code($denied ? 403 : 401);
```

7. Validation and Security

- **Server-side validation:** every field is checked by `validate_student_record()`; the form has `novalidate`, so the browser never decides. Invalid input is answered with HTTP 422, a message under each field and the general alert "Please correct the highlighted fields. Nothing was saved."
- **Duplicates:** checked before saving (all errors at once), again inside the transaction, and finally by the UNIQUE keys on `enrollment_no` and `email` – also in another letter case and with extra spaces.
- **SQL injection:** all queries with values are prepared statements; filter values are accepted only from fixed lists; the page number and record IDs are matched against strict patterns; LIKE wildcards are escaped.
- **XSS:** every value is printed through `e()` = `htmlspecialchars(…, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`.

- **CSRF:** every POST form carries the session's 64-character token, compared with `hash_equals()`; a missing, wrong or modified token (or another session's token) is refused with HTTP 403 "Security check failed" and logged as CSRF_REJECTED. GET requests – even with action parameters and the token – change nothing.
- **Authorisation:** `require_permission('students.manage')` on the export and on every action; students, editors and visitors are refused on the server, not only hidden in the menu.
- **CSV export:** formula-injection guard, RFC 4180 quoting by `fputcsv()`, no account or session data.
- **Records are never lost:** the code contains no `DELETE FROM students`; deactivation is reversible.
- **Headers:** signed-in pages send `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin` and `Cache-Control: no-store`.
- **Deactivated students:** refused at sign-in after password verification and signed out at the next request (section 6.5).

8. Output Screenshots

All screenshots are genuine full-page captures from Microsoft Edge (headless, driven by Playwright, window 1280 px wide) of the integrated portal running on PHP's built-in server with a disposable database built from `database/studenthub_demo.sql` plus fictional students added by the test. Figures 1–11 were produced by `tests/browser_students.py` (screenshots 01–13), Figures 12–15 by `tests/browser_inactive_login.py` (screenshots 14–17). Two further captures of the folder are not reproduced here: `02-add-student-form.png` (the empty add form, the same form as in Figure 2) and `08-student-updated.png` (the list after saving the edit of Figure 6). The Admin Console gained further tabs in later integration steps (for example *Inquiries*, Practical 7), which are therefore not visible in these screenshots.

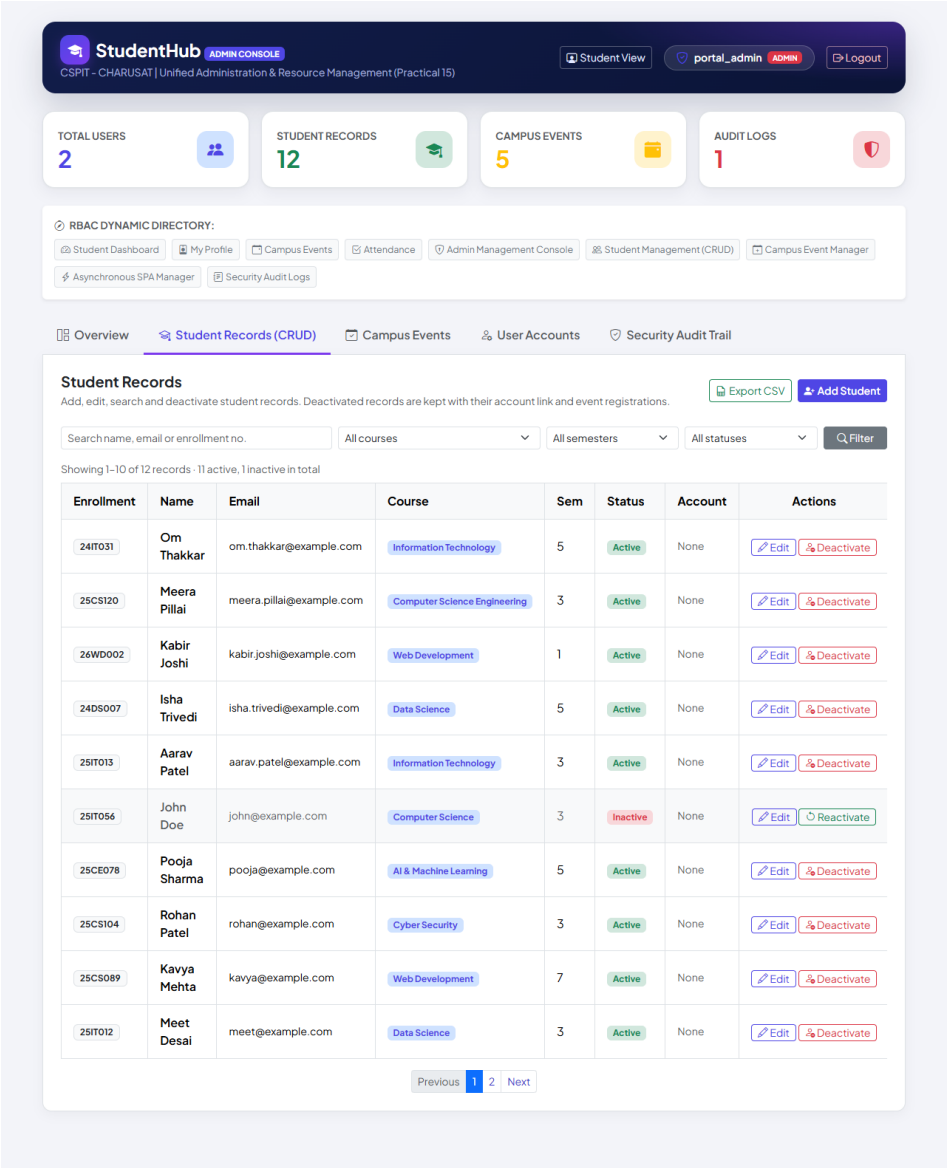


Figure 1: Student Records tab of the Admin Console (administrator portal_admin): 10 of 12 records on page 1 with enrollment number, name, e-mail, course, semester, status and account columns; John Doe (25IT056) is Inactive with a greyed row and a Reactivate button; summary "11 active, 1 inactive in total" and pagination (01-student-list.png)

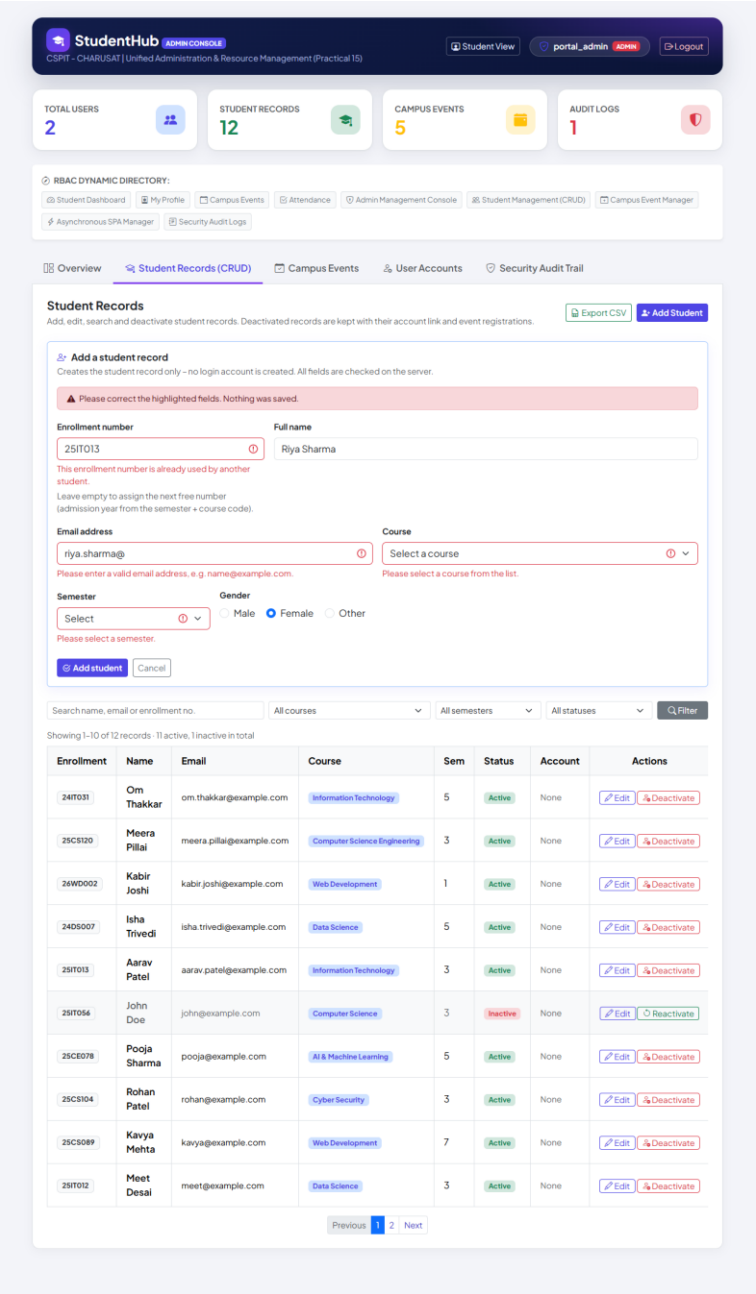


Figure 2: Server-side validation of the add form: the enrollment number 25IT013 is already used by another student, the e-mail "riya.sharma@" is invalid, no course and no semester were selected; the typed name Riya Sharma is kept and nothing was saved (03-validation-error.png)

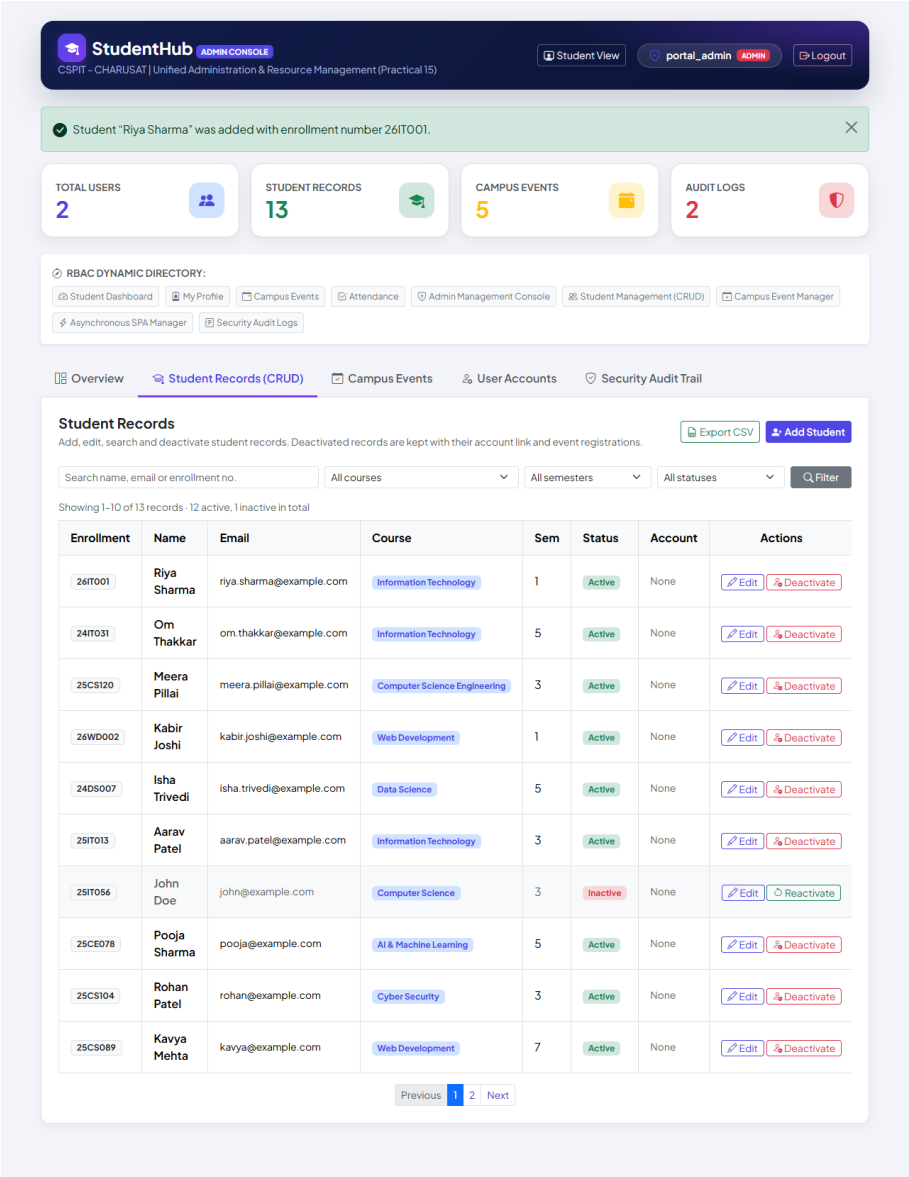


Figure 3: After a valid submission with an empty enrollment number: "Student “Riya Sharma” was added with enrollment number 26IT001." – the number was assigned automatically and the new record is first in the list (13 records) (04-student-created.png)

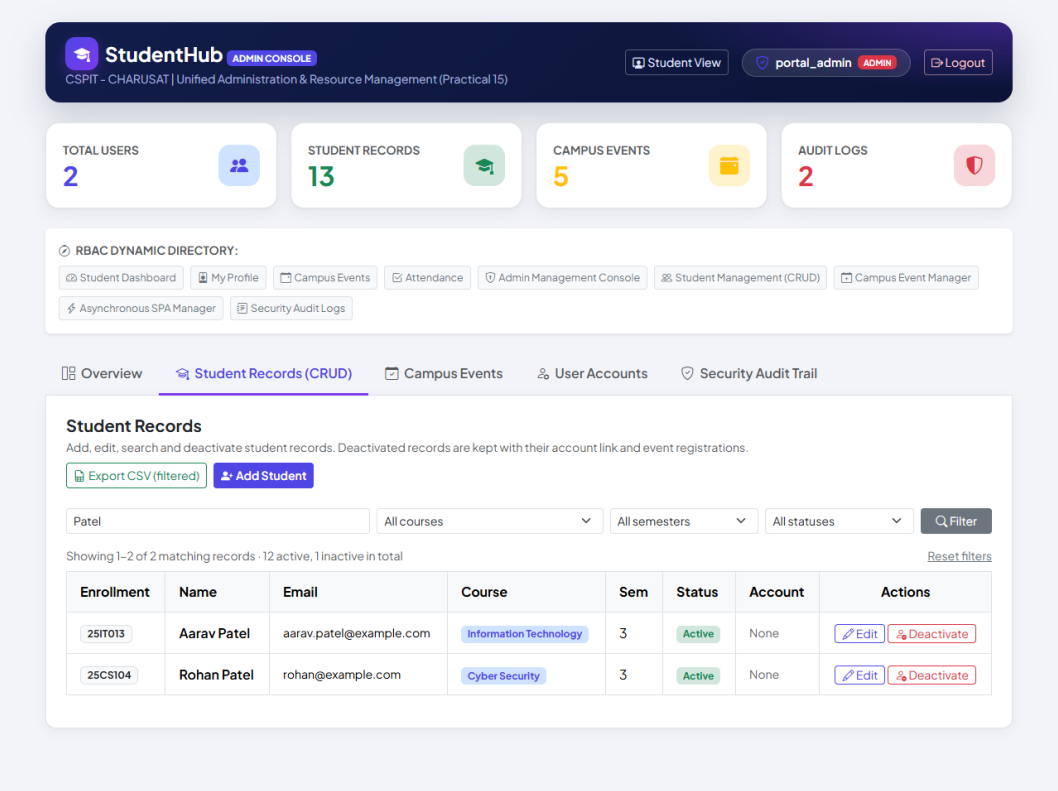


Figure 4: Search "Patel": 2 matching records (Aarav Patel, Rohan Patel); the export button now reads "Export CSV (filtered)" and a "Reset filters" link is shown (05-search-results.png)

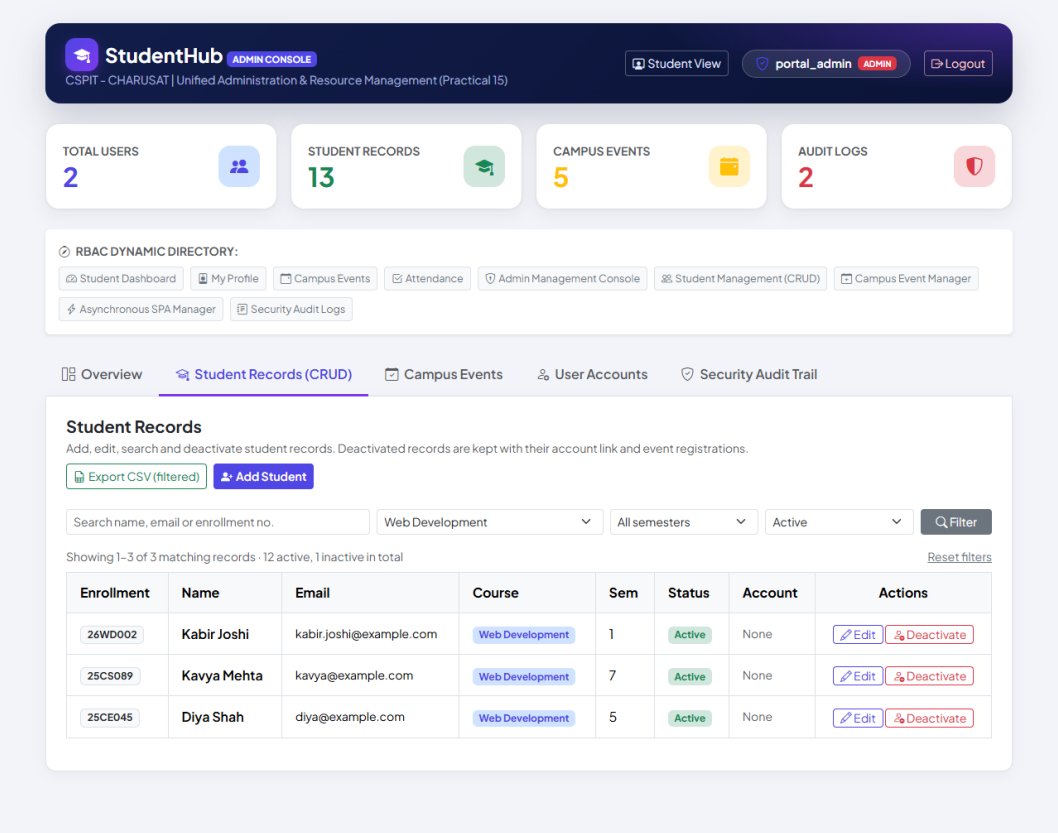


Figure 5: Combined filters course "Web Development" + status "Active": the 3 matching records (Kabir Joshi, Kavya Mehta, Diya Shah); the selected filters stay in the form (06-filtered-records.png)

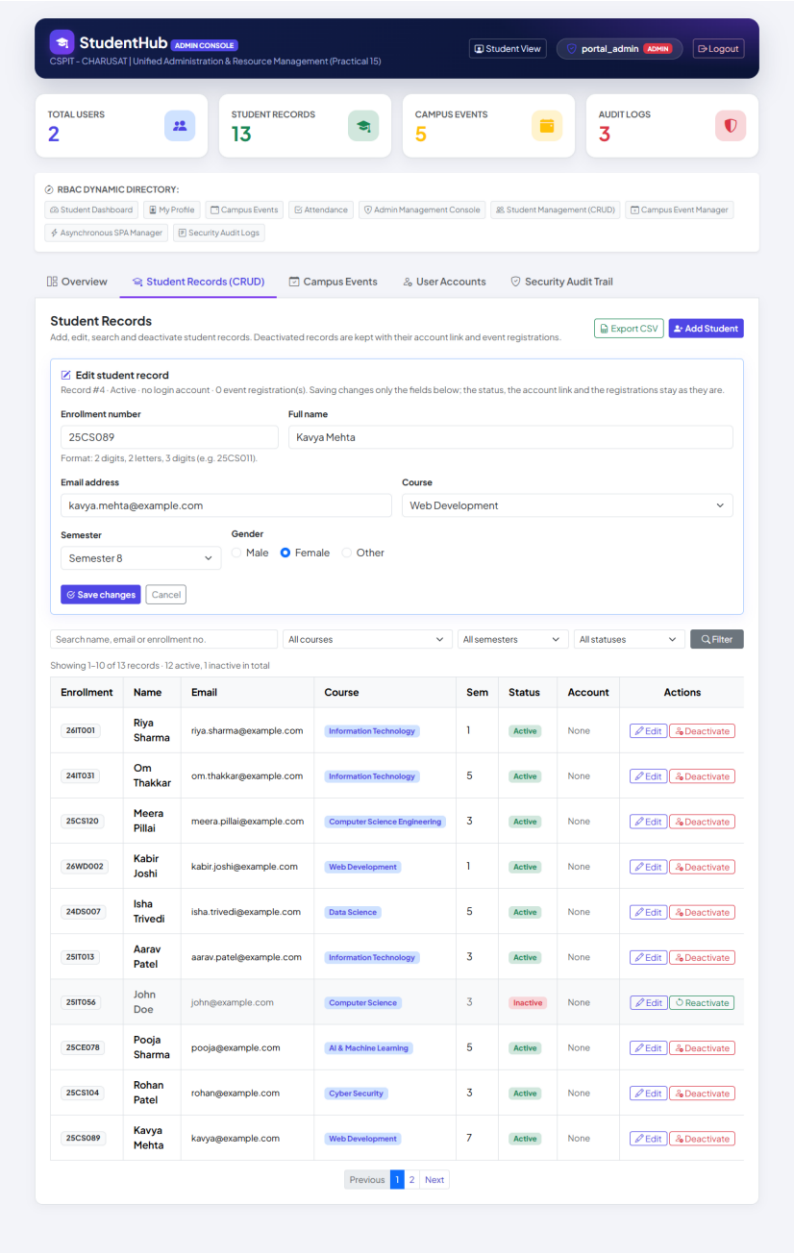


Figure 6: Edit form for Kavya Mehta filled from the database: "Record #4 · Active · no login account · 0 event registration(s)"; the e-mail is being changed to kavya.mehta@example.com and the semester to 8 (07-edit-student.png)

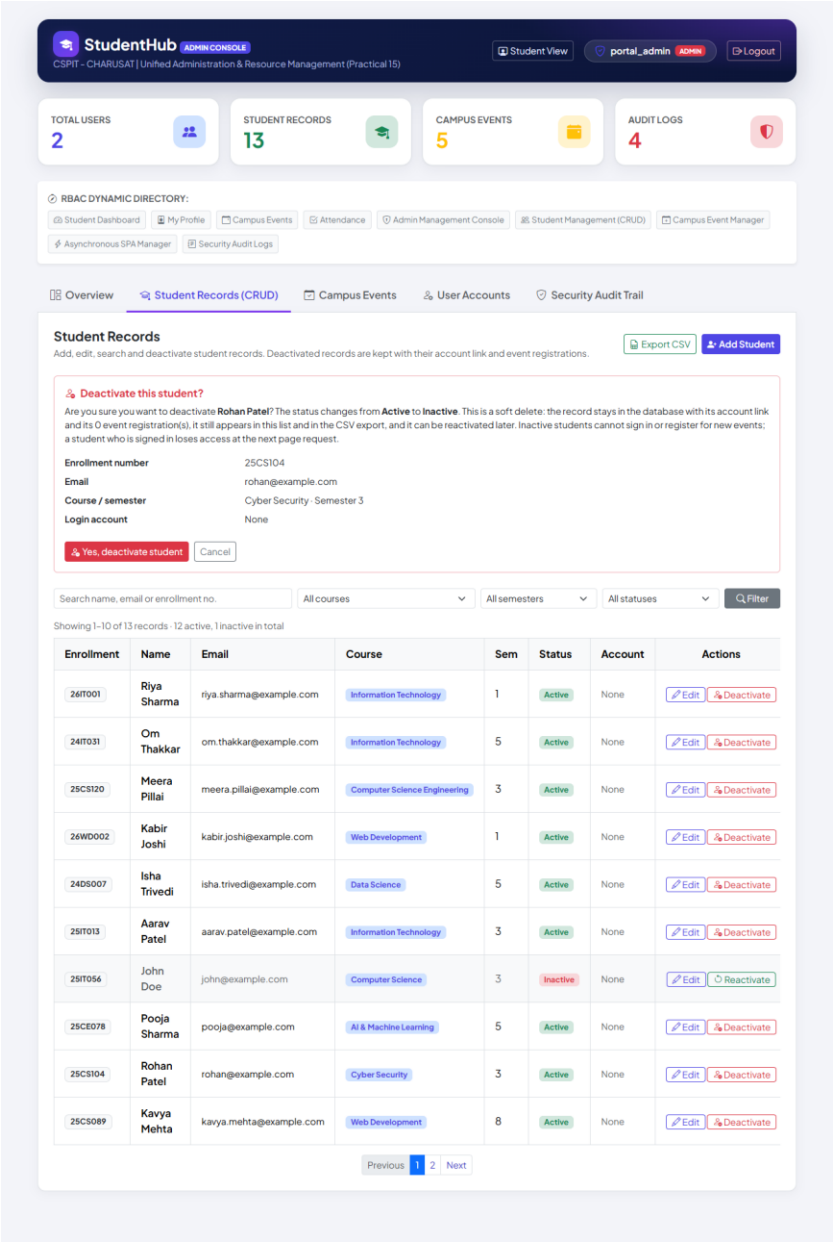


Figure 7: Confirmation before the soft delete of Rohan Patel (25CS104): the panel explains that the record, its account link and its registrations are kept, that it stays in the list and the CSV export, and that inactive students cannot sign in or register; nothing has changed yet (09-deactivate-confirmation.png)

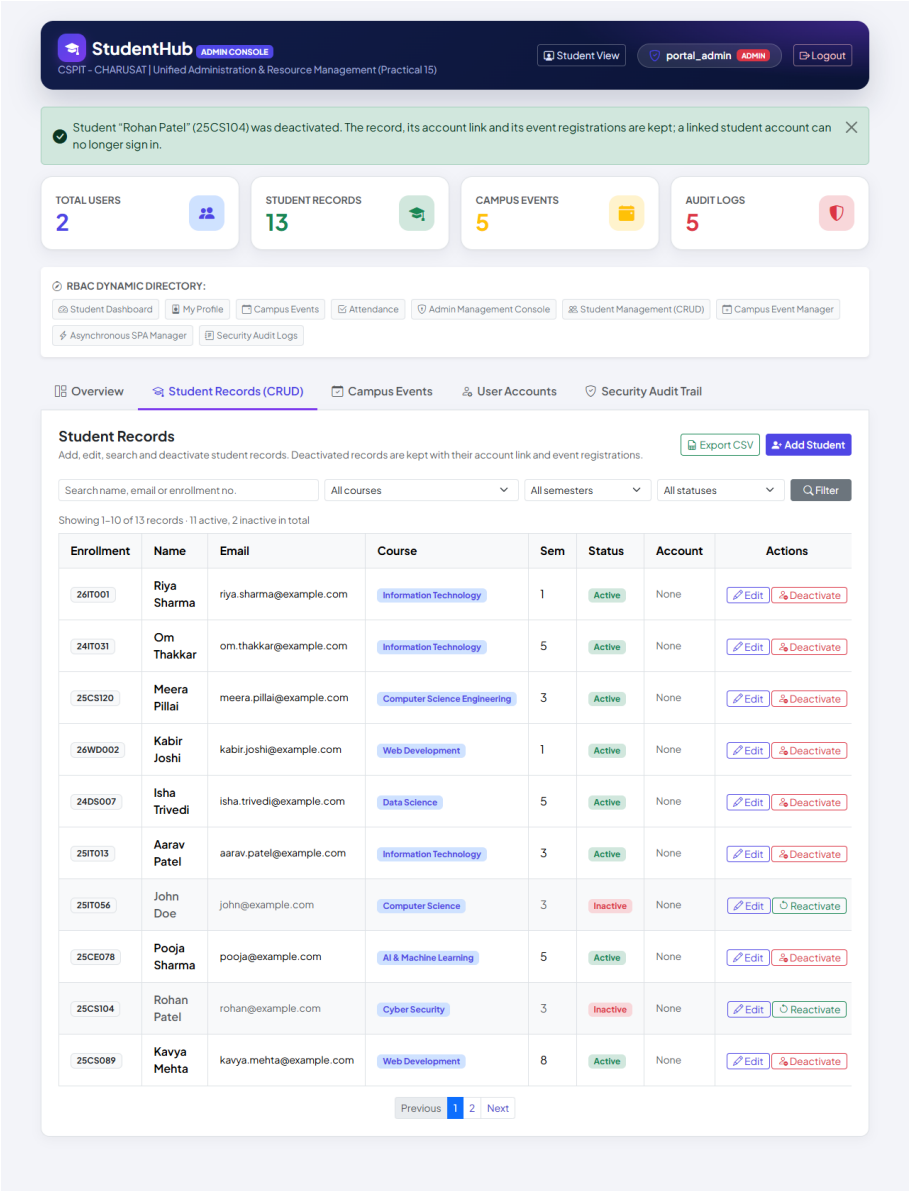


Figure 8: After the confirmed deactivation: "Student "Rohan Patel" (25CS104) was deactivated. The record, its account link and its event registrations are kept; a linked student account can no longer sign in." – the row is still listed, greyed, with an Inactive badge and a Reactivate button (11 active, 2 inactive, still 13 records) (10-student-inactive.png)

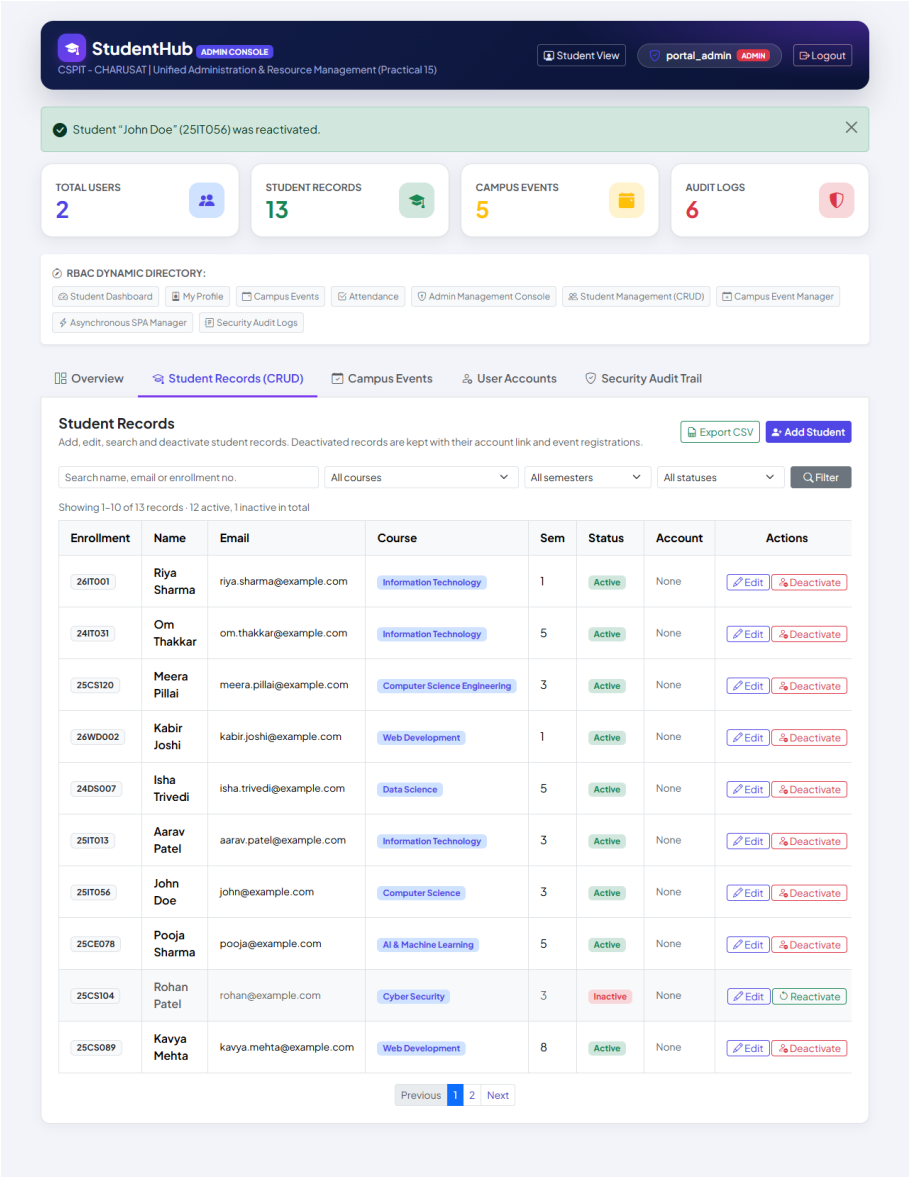


Figure 9: Reactivation: "Student "John Doe" (25IT056) was reactivated." – John Doe is Active again (12 active, 1 inactive) (11-student-reactivated.png)

Downloaded file: studenthub_students_20261031_012936.csv

Export of the filtered list (course Web Development, status Active) from the StudentHub Admin Console · 390 bytes, UTF-8 with byte-order mark, CRLF line endings. Raw file content:

```
"Record ID","Enrollment No",Name,Email,Course,Semester,Status,"Login Account","Created At"
10,26WD002,"Kabir Joshi",kabir.joshi@example.com,"Web Development",1,Active,None,"2026-10-31 01:29:33"
4,25CS089,"Kavya Mehta",kavya@example.com,"Web Development",7,Active,None,"2026-10-31 01:29:32"
2,25CE045,"Diya Shah",diya@example.com,"Web Development",5,Active,None,"2026-10-31 01:29:32"
```

The same file read as CSV (columns):

Record ID	Enrollment No	Name	Email	Course	Semester	Status	Login Account	Created At
10	26WD002	Kabir Joshi	kabir.joshi@example.com	Web Development	1	Active	None	2026-10-31 01:29:33
4	25CS089	Kavya Mehta	kavya@example.com	Web Development	7	Active	None	2026-10-31 01:29:32
2	25CE045	Diya Shah	diya@example.com	Web Development	5	Active	None	2026-10-31 01:29:32

Figure 10: The CSV file downloaded by the Export button of Figure 5's filtered view (course Web Development, status Active): studenthub_students_20261031_012936.csv, 390 bytes, UTF-8 with byte-order mark, CRLF line endings. A headless browser has no download bar, so the test saved the download (also kept as 12-csv-export-web-development-active.csv) and displayed its exact content as text and, parsed as CSV, as a table in the browser; Excel was not used (12-csv-export.png)

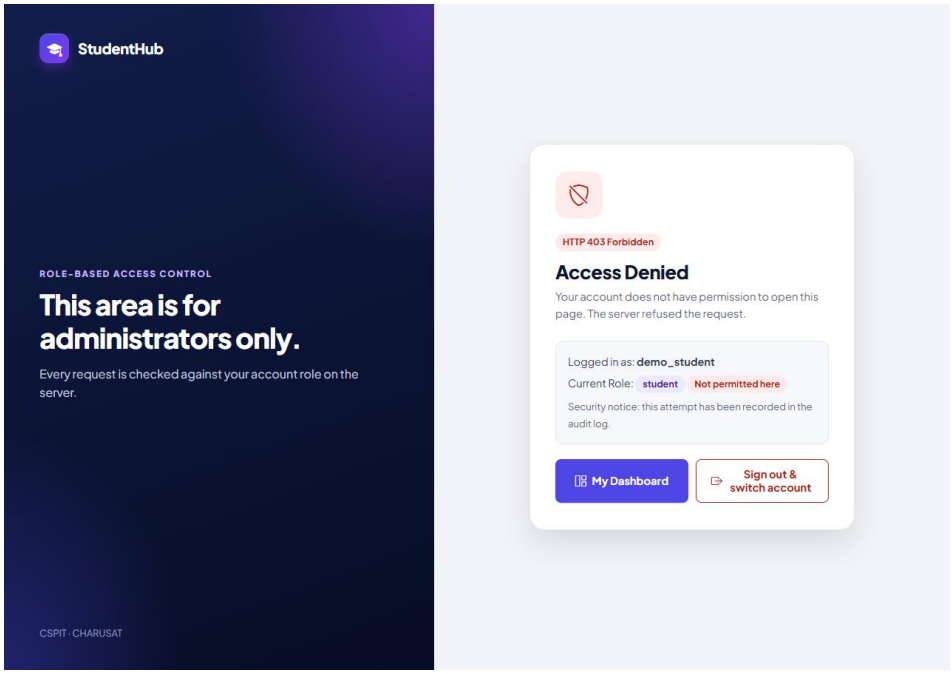


Figure 11: A signed-in student (demo_student) opening the Student Records tab: HTTP 403 "Access Denied" page, role student "Not permitted here", no student data shown; the attempt is recorded in the audit log (13-student-access-denied.png)

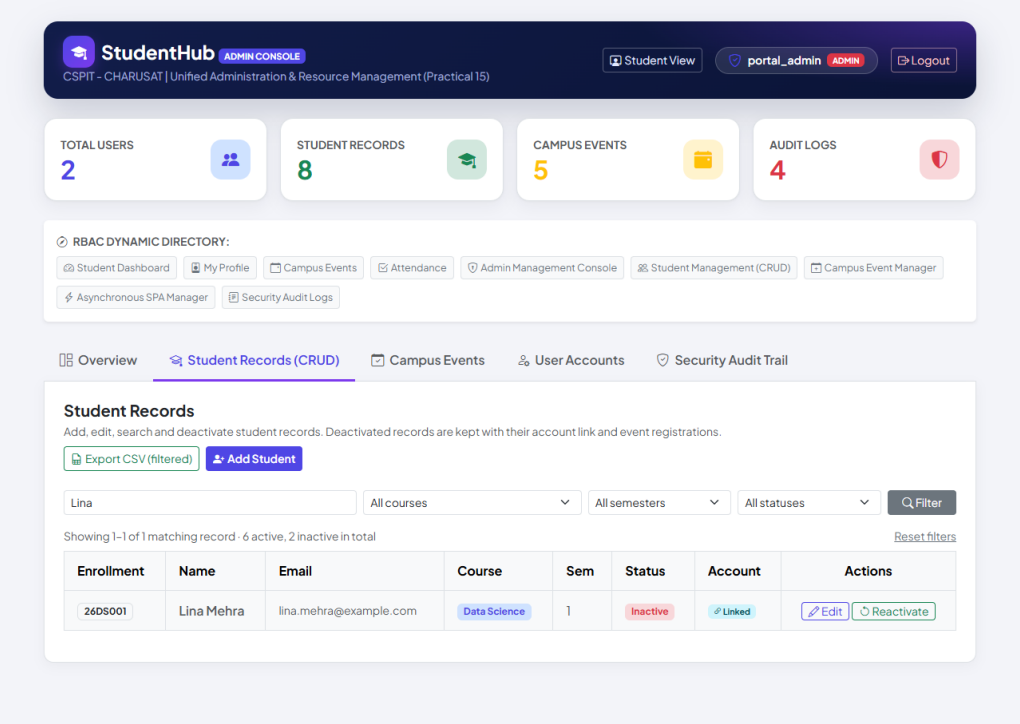


Figure 12: Sign-in fix: Lina Mehra registered herself (record 26DS001, account column "Linked") and the administrator has deactivated the record – status Inactive, Reactivate button (14-admin-deactivated-linked-student.png)

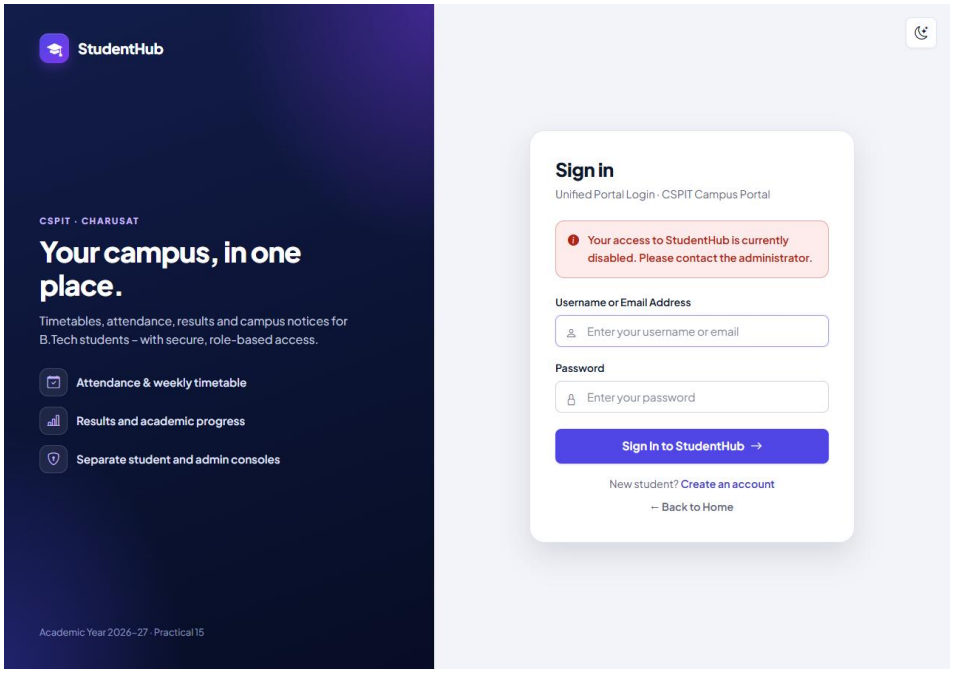


Figure 13: The student's already open session: her next page request leads to the sign-in page with the generic message "Your access to StudentHub is currently disabled. Please contact the administrator." (15-session-ended-after-deactivation.png)

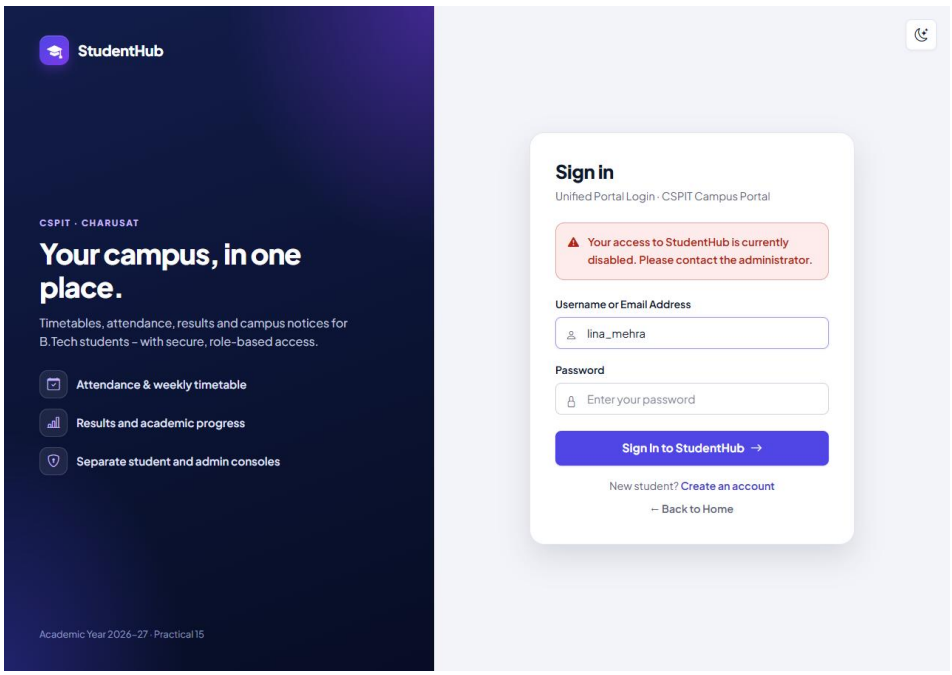


Figure 14: Signing in again as lina_mehra with the correct password is refused with the same generic message (no record details); no session is started (16-inactive-login-denied.png)

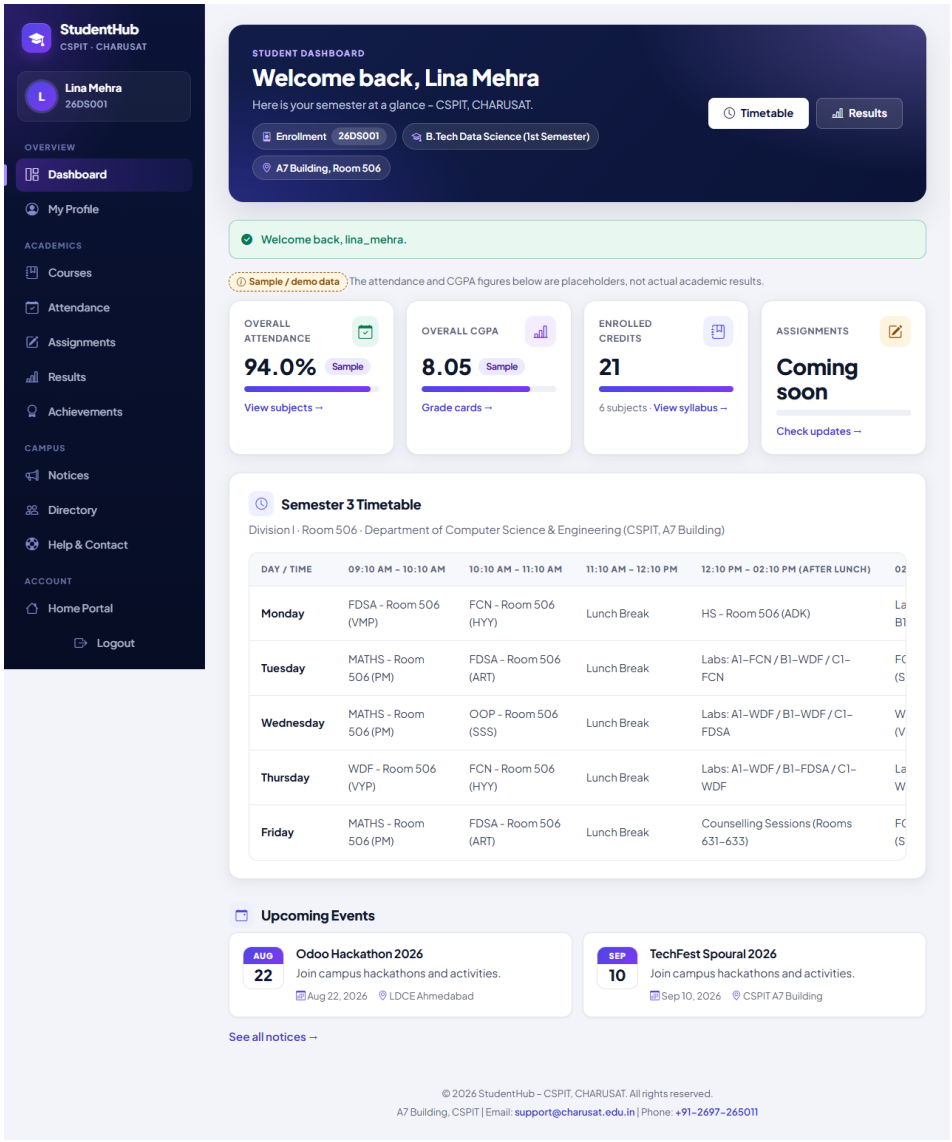


Figure 15: After the administrator reactivated the record the student signs in normally: dashboard with "Welcome back, Lina Mehra", enrollment 26DS001 and the message "Welcome back, lina_mehra." (17-login-after-reactivation.png)

9. Testing and Results

All tests were executed on 31 October 2026 on Windows 11 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and PHP's built-in server, against disposable databases (built from `database/studenthub_demo.sql` or from a restored copy of the real database, dropped afterwards). The live `studenthub_db` received only migration 002, after a verified backup, and its users, password hashes, students, events, registrations and menu items were verified unchanged afterwards. The numbers below are copied from the saved outputs and from TEST_REPORT.md §7–§8.

Suite	Result	Output
`tests/test_students.py` – HTTP tests of the Student Records tab (demo data)	**170 / 170 passed**	`tests/test_students_output.txt`
The same with `--source-dump` (restored copy of the real database)	**170 / 170 passed**	not committed (contains real records)

tests/browser_students.py – Microsoft Edge + screenshots 01–13	**16 / 16 passed**	tests/browser_students_output.txt
tests/browser_inactive_login.py – Edge: register, deactivate, session ended, sign-in refused, reactivate	**13 / 13 passed**	tests/browser_inactive_login_output.txt
database/tests/test_002_unique_enrollment_no.py – fixture / real dump	**22 / 22** and **22 / 22 passed**	database/tests/test_002_output.txt
test_002 … --verify-live – live database after migration 002 (read-only)	**4 / 4 passed**	database/tests/test_002_verify_live_output.txt
Regression: tests/test_auth.py (Practicals 9/10) / tests/browser_auth.py	**132 / 132** / **14 / 14 passed**	tests/test_auth_output.txt, tests/browser_auth_output.txt

Regression of the Practical 8 migration test (database/tests/test_001_practical8_foundation.py): TEST_REPORT §7–§8 record 87 / 87 (fixture) and 24 / 24 (--verify-live) for this step. The saved file database/tests/test_001_output.txt now holds a later re-run (88 / 88).

9.1 Checks per area (tests/test_students.py)

Area	Checks	Examples of what is verified
Static checks	5	php -l on all PHP files of studenthub/ and tools/; no DELETE FROM students; prepared statements; students.manage on the server
Database (migration 002)	4	UNIQUE index present, 002 recorded, duplicate refused also in another letter case, several NULLs allowed
Student listing (admin)	7	newest first, columns, summary 6 active / 1 inactive, Inactive marking, CSRF token in every form, empty state
Add student	11	entered and automatic numbers (26IT102, 25DS001), normalisation, no login account created, CREATE_STUDENT
Invalid student details	14	empty / digits / HTML / one-letter name, invalid or too long e-mail, unknown course or gender, SQL in the number, array field
Duplicates	5	same number, lower case, with spaces; e-mail in other case; duplicate reported together with other errors
Invalid semester values	9	0, 9, 12, -1, 3.5, abc, 1 OR 1=1, empty – all refused, nothing stored
Edit student	14	values saved; link, status, number, creation time and registrations unchanged; legacy record editable; unknown IDs
Search and filters	14	name, e-mail, number prefix, course, semester, status and combinations; % and _ literal
SQL injection attempts	9	four injection strings in the search; injected course, semester, status and page values ignored
Pagination	5	24 records → 3 pages of 10; last page; page past the end; filters kept

Soft delete and reactivation	21	confirmation page, GET / token-less requests change nothing, record and registrations kept, `student_inactive`, reactivation
Inactive records elsewhere	2	hidden from the student directory, listed again after reactivation
CSV export	14	BOM, CRLF, header, all 27 records incl. inactive, formula injection, 9 columns, filtered export of all pages, file-name time zone
Access control	7	students, editors and visitors refused for tab, forms, export and POST actions
CSRF and GET requests	4	missing, wrong, modified and another admin's token refused and logged
Deactivated students cannot sign in	22	session ended (new ID, SESSION_REVOKED), old cookie useless, sign-in 403 + LOGIN_DENIED, editors/admins unaffected, reactivation
Server log / clean-up	3	no PHP warnings; disposable database removed; live database untouched
Total	**170**	

9.2 Browser checks (Microsoft Edge)

`tests/browser_students.py` (16 checks) clicked through the tab as a user would: list and pagination, Inactive badge, add form, server-side validation messages, successful add with the assigned number, search *Patel*, combined filters, the CSV download (file name pattern, BOM, header, only the filtered rows), edit and save, deactivation with confirmation, reactivation, HTTP 403 for a student, no JavaScript errors and no PHP warnings. `tests/browser_inactive_login.py` (13 checks) registered a new student, deactivated the linked record in a second browser context, and verified that the student's open session ended, that a new sign-in was refused with the generic message, and that sign-in worked again after reactivation while the administrator's session was never affected. Both outputs report one known, pre-existing 404 for `/favicon.ico` (the portal has no favicon).

10. Problems Found and Fixed During Integration

- **Time zone of dates:** XAMPP's PHP used the `Europe/Berlin` time zone from php.ini while MariaDB uses the system's India time, so between midnight and about 04:30 the portal's dates (form defaults, academic year for enrollment numbers, CSV file name) were one day behind. `studenthub/db.php` now sets `Asia/Kolkata` (override: `STUDENTHUB_TIMEZONE`); a test checks that the export file name matches the database clock.
- **Deactivated students could still sign in:** found after the first integration commit (`0ac2f5d`) and listed as a limitation; fixed in commit `6420efc` (section 6.5) with 22 new HTTP checks and the Edge test `browser_inactive_login.py`. The confirmation panel and the success message now say that a linked student account can no longer sign in, and screenshots 01–13 were regenerated with the new wording.
- **All problems reported at once:** field validation needs no database, the duplicate checks do. `check_student_record()` runs both and merges the results, so a duplicate enrollment number is reported together with the other errors (Figure 2) instead of only after the other errors have been corrected.
- **CSV evidence in a headless browser:** a headless Edge has no download bar, so the test saved the downloaded file and showed its bytes as text and as a table (Figure 10) instead of a screenshot of Excel.

11. Differences from the Original Standalone Practical and Limitations

Aspect	Original report (Practical-11/)	Integrated portal (studenthub/)
Database	Own database `studenthub_prac11`, table `students` (student_id, full_name, …,	Shared `studenthub_db.students` – the same rows used by self-registration, profiles and event

	status ENUM('active','inactive'), deleted_at)	registrations
Access	Standalone application without a login	Admin Console behind the portal login; `students.manage` (administrators only)
CSRF	One-time form tokens	One CSRF token per session on every POST, checked with `hash_equals()`
Search / filters	Name, e-mail and course; course and status filter	Name, e-mail and enrollment number; course, semester and status filters
Soft delete	status inactive + `deleted_at` time	status Inactive; time in the audit log; Reactivate button
Enrollment numbers	–	YYPPnnn scheme, automatic next number, UNIQUE index (migration 002)
Sign-in effect	–	A linked student account cannot sign in while its record is Inactive
CSV file	`students_YYYYMMDD_HHMMSS.csv`, 9 columns incl. Deactivated At	`studenthub_students_YYYYMMDD_HHMMSS.csv`, 9 columns (Record ID … Login Account, Created At)
Tests	106 / 106 automated, 13 / 13 browser	170 / 170 automated, 16 / 16 + 13 / 13 browser, migration tests

Limitations (TEST_REPORT §7): there is no interface to link an added record to an existing login account (links are created only by self-registration); the soft-delete time is recorded in the audit log, not in a column of `students`; the add form offers automatic enrollment numbers only for the four registration courses (Information Technology, Computer Science Engineering, Data Science, Web Development) – other courses need a typed number; the Admin Console still loads Bootstrap from a CDN; the browser reports one 404 for `/favicon.ico`.

12. Conclusion

Student management now works on the portal's own student records instead of a separate database. Administrators list, search and filter the records, add and edit them with full server-side validation and unique enrollment numbers, soft-delete them only after a confirmation and reactivate them, and export exactly the filtered records to a CSV file protected against formula injection. Deactivation never destroys data – the record, its account link and its event registrations stay – and a deactivated student can neither sign in nor keep an open session. Prepared statements, output escaping, a CSRF token on every POST, server-side permission checks and audit logging protect the tab. 170 automated checks, 16 + 13 browser checks and the migration tests passed.

13. References

- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – `studenthub/` (portal), `database/` (schema and migration 002), `tests/`, `evidence/practical-11-integration/`, `TEST_REPORT.md` §7–§8; commits `0ac2f5d`, `6420efc`
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – htmlspecialchars: https://www.php.net/manual/en/function.htmlspecialchars.php
- PHP Manual – fputcsv: https://www.php.net/manual/en/function.fputcsv.php
- PHP Manual – hash_equals: https://www.php.net/manual/en/function.hash-equals.php
- PHP Manual – password_verify: https://www.php.net/manual/en/function.password-verify.php
- OWASP – CSV Injection: https://owasp.org/www-community/attacks/CSV_Injection

- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP – SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html